MINOR ASSIGNMENT-1: OBJECT-ORIENTED PROGRAMMING (OOP)

1. What is the significance of classes in Python programming, and how do they contribute to objectoriented programming?

Enable object-oriented programming: encapsulation, inheritance, polymorphism

Help structure code into reusable components

Allow bundling of data + behavior

Improve maintainability and scalability

In [2]:
```
class Example:
    def __init__(self, value):
        self.value = value
    def display(self):
        return self.value

obj = Example(10)
print(obj.display())
"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
10
```

Out[2]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

2. Create a custom Python class for managing a bank account with basic functionalities like deposit and withdrawal?

In [5]:
```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient balance"
        self.balance -= amount
    def display(self):
        return self.balance

acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(300)
print(acc.display())
"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
1200
```

Out[5]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

3. Create a Book class that contains multiple Chapters, where each Chapter has a title and page count. Write code to initialize a Book object with three chapters and display the total page count of the book.

In [6]:
```
class Chapter:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

class Book:
    def __init__(self, chapters):
        self.chapters = chapters
    def total_pages(self):
        return sum(c.pages for c in self.chapters)

c1 = Chapter("Intro", 10)
c2 = Chapter("Middle", 30)
c3 = Chapter("End", 20)

book = Book([c1, c2, c3])
print(book.total_pages())
"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
60
```

Out[6]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

4. How does Python enforce access control to class attributes, and what is the difference between public, pmrotected, and private attributes?

In [9]:
```
class Test:
    def __init__(self):
        self.public = 1
        self._protected = 2
        self.__private = 3

obj = Test()
print(obj.public)
print(obj._protected)
try:
    print(obj.__private)
except:
    print("Private not accessible")

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
1
2
Private not accessible
```

Out[9]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

5. Write a Python program using a Time class to input a given time in 24-hour format and convert it to a 12-hour format with AM/PM. The program should also validate time strings to ensure they are in the correct HH:MM:SS format. Implement a method to check if the time is valid and return an appropriate message

In [10]:
```
class Time:
    def __init__(self, time_str):
        self.time_str = time_str

    def is_valid(self):
        parts = self.time_str.split(":")
        if len(parts) != 3:
            return False
        h, m, s = parts
        if not (h.isdigit() and m.isdigit() and s.isdigit()):
            return False
        h, m, s = int(h), int(m), int(s)
        return 0 <= h < 24 and 0 <= m < 60 and 0 <= s < 60

    def to_12_hour(self):
        if not self.is_valid():
            return "Invalid"
        h, m, s = map(int, self.time_str.split(":"))
        period = "AM" if h < 12 else "PM"
        h = h % 12 or 12
        return f"{h:02}:{m:02}:{s:02} {period}"

t = Time("14:30:45")
print(t.is_valid())
print(t.to_12_hour())

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
True
02:30:45 PM
```

Out[10]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

6. Write a Python program that uses private attributes for creating a BankAccount class. Implement methods to deposit, withdraw, and display the balance, ensuring direct access to the balance attribute is restricted. Explain why using private attributes can help improve data security and prevent accidental modifications

In [11]:
```
class BankAccount:
    def __init__(self, balance=0):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount > self.__balance:
            return "Insufficient"
        self.__balance -= amount

    def display(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(200)
print(acc.display())
```

```
"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
1300
```

Out[11]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

7. Write a Python program to simulate a card game using object-oriented principles. The program should include a Card class to represent individual playing cards, a Deck class to represent a deck of cards, and a Player class to represent players receiving cards. Implement a shuffle method in the Deck class to shuffle the cards and a deal method to distribute cards to players. Display each player's hand after dealing.

In [12]:
```
import random

class Card:
    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank
    def __repr__(self):
        return f"{self.rank} of {self.suit}"

class Deck:
    def __init__(self):
        suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
        ranks = ["2","3","4","5","6","7","8","9","10","J","Q","K","A"]
        self.cards = [Card(s, r) for s in suits for r in ranks]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, players, n=5):
        for _ in range(n):
            for p in players:
                p.hand.append(self.cards.pop())

class Player:
    def __init__(self, name):
        self.name = name
        self.hand = []

deck = Deck()
deck.shuffle()

p1 = Player("A")
p2 = Player("B")

deck.deal([p1, p2])

print(p1.hand)
print(p2.hand)

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
[5 of Diamonds, 7 of Spades, 2 of Diamonds, A of Hearts, 10 of Diamonds]
[A of Clubs, 10 of Clubs, 2 of Clubs, 4 of Clubs, 4 of Hearts]
```

Out[12]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

8. Write a Python program that defines a base class Vehicle with attributes make and model, and a method display info(). Create a subclass Car that inherits from Vehicle and adds an additional attribute num doors. Instantiate both Vehicle and Car objects, call their display info() methods, and explain how the subclass inherits and extends the functionality of the base class.

In [13]:
```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model
    def display_info(self):
        return f"{self.make} {self.model}"

class Car(Vehicle):
    def __init__(self, make, model, doors):
        super().__init__(make, model)
        self.doors = doors
    def display_info(self):
        return f"{self.make} {self.model}, Doors: {self.doors}"

v = Vehicle("Toyota", "Base")
c = Car("Honda", "City", 4)

print(v.display_info())
print(c.display_info())

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
Toyota Base
Honda City, Doors: 4
```

Out[13]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

9. Write a Python program demonstrating polymorphism by creating a base class Shape with a method area(), and two subclasses Circle and Rectangle that override the area() method. Instantiate objects of both subclasses and call the area() method. Explain how polymorphism simplifies working with different shapes in an inheritance hierarchy.

In [14]:
```
import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return math.pi * self.r * self.r

class Rectangle(Shape):
    def __init__(self, l, b):
```

```
        self.l = l
        self.b = b
    def area(self):
        return self.l * self.b

shapes = [Circle(3), Rectangle(4,5)]

for s in shapes:
    print(s.area())

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
28.274333882308138
20
```

Out[14]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

10. Implement the CommissionEmployee class with init , earnings, and repr methods. Include properties for personal details and sales data. Create a test script to instantiate the object, display earnings, modify sales data, and handle data validation errors for negative values.

In [15]:
```
class CommissionEmployee:
    def __init__(self, name, sales, rate):
        if sales < 0 or rate < 0:
            raise ValueError("Invalid")
        self.name = name
        self.sales = sales
        self.rate = rate

    def earnings(self):
        return self.sales * self.rate

    def __repr__(self):
        return f"{self.name}, Earnings: {self.earnings()}"

    def set_sales(self, sales):
        if sales < 0:
            raise ValueError("Invalid")
        self.sales = sales

emp = CommissionEmployee("John", 10000, 0.1)
print(emp)
emp.set_sales(20000)
print(emp.earnings())

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
John, Earnings: 1000.0
2000.0
```

Out[15]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

11. What is duck typing in Python? Write a Python program demonstrating duck typing by creating a function describe() that accepts any object with a speak() method. Implement two classes, Dog and Robot, each with a speak() method. Pass instances

of both classes to the describe() function and explain how duck typing allows the function to work without checking the object's type

In [16]:
```
class Dog:
    def speak(self):
        return "Bark"

class Robot:
    def speak(self):
        return "Beep"

def describe(obj):
    return obj.speak()

d = Dog()
r = Robot()

print(describe(d))
print(describe(r))

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
Bark
Beep
```

Out[16]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

12. WAP to overload the + operator to perform addition of two complex numbers using a custom Complex class?

In [17]:
```
class Complex:
    def __init__(self, r, i):
        self.r = r
        self.i = i

    def __add__(self, other):
        return Complex(self.r + other.r, self.i + other.i)

    def __repr__(self):
        return f"{self.r}+{self.i}i"

c1 = Complex(2,3)
c2 = Complex(4,5)

print(c1 + c2)

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
6+8i
```

Out[17]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

13. WAP to create a custom exception class in Python that displays the balance and withdrawal amount when an error occurs due to insufficient funds?

In [18]:
```
class InsufficientFunds(Exception):
    def __init__(self, balance, amount):
```

```
        self.balance = balance
        self.amount = amount
    def __str__(self):
        return f"Balance: {self.balance}, Withdrawal: {self.amount}"

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFunds(self.balance, amount)
        self.balance -= amount

acc = BankAccount(1000)

try:
    acc.withdraw(1500)
except Exception as e:
    print(e)

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
Balance: 1000, Withdrawal: 1500
```

Out[18]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

14. Write a Python program using the Card data class to simulate dealing 5 cards to a player from a shuffled deck of standard playing cards. The program should print the player's hand and the number of remaining cards in the deck after the deal.

In [19]:
```
import random
from dataclasses import dataclass

@dataclass
class Card:
    suit: str
    rank: str

suits = ["Hearts","Diamonds","Clubs","Spades"]
ranks = ["2","3","4","5","6","7","8","9","10","J","Q","K","A"]

deck = [Card(s,r) for s in suits for r in ranks]
random.shuffle(deck)

hand = [deck.pop() for _ in range(5)]

print(hand)
print(len(deck))

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
[Card(suit='Diamonds', rank='9'), Card(suit='Diamonds', rank='10'), Card(suit='Cl
ubs', rank='8'), Card(suit='Diamonds', rank='A'), Card(suit='Diamonds', rank
='J')]
47
```

Out[19]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

15. How do Python data classes provide advantages over named tuples in terms of flexibility and functionality? Give an example using python code.

In [20]:
```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int

p = Person("Alice", 25)
p.age = 30

print(p)

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52"""
```

```
Person(name='Alice', age=30)
```

Out[20]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

16. Write a Python program that demonstrates unit testing directly within a function's docstring using the doctest module. Create a function add(a, b) that returns the sum of two numbers and includes multiple test cases in its docstring. Implement a way to automatically run the tests when the script is executed.

In [21]:
```
import doctest

def add(a, b):
    """
    >>> add(2, 3)
    5
    >>> add(-1, 1)
    0
    >>> add(0, 0)
    0
    """
    return a + b

doctest.testmod()

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52"""
```

Out[21]:
```
'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'
```

17. Scope Resolution: object's namespace → class namespace → global namespace → built-in namespace.

What will be the output when the above program is executed? Explain the scope resolution process step by step.

In [22]:
```
species = "Global Species"

class Animal:
```

```
    species = "Class Species"
    def __init__(self, species):
        self.species = species
    def display_species(self):
        print("Instance species:", self.species)
        print("Class species:", Animal.species)
        print("Global species:", globals()['species'])

a = Animal("Instance Species")
a.display_species()

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
Instance species: Instance Species
Class species: Class Species
Global species: Global Species
```

Out[22]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'

18. Write a Python program using a lambda function to convert temperatures from Celsius to Kelvin, store the data in a tabular format using pandas, and visualize the data using a plot.

In [23]:
```
import pandas as pd
import matplotlib.pyplot as plt

celsius = [0, 10, 20, 30, 40]

kelvin = list(map(lambda c: c + 273.15, celsius))

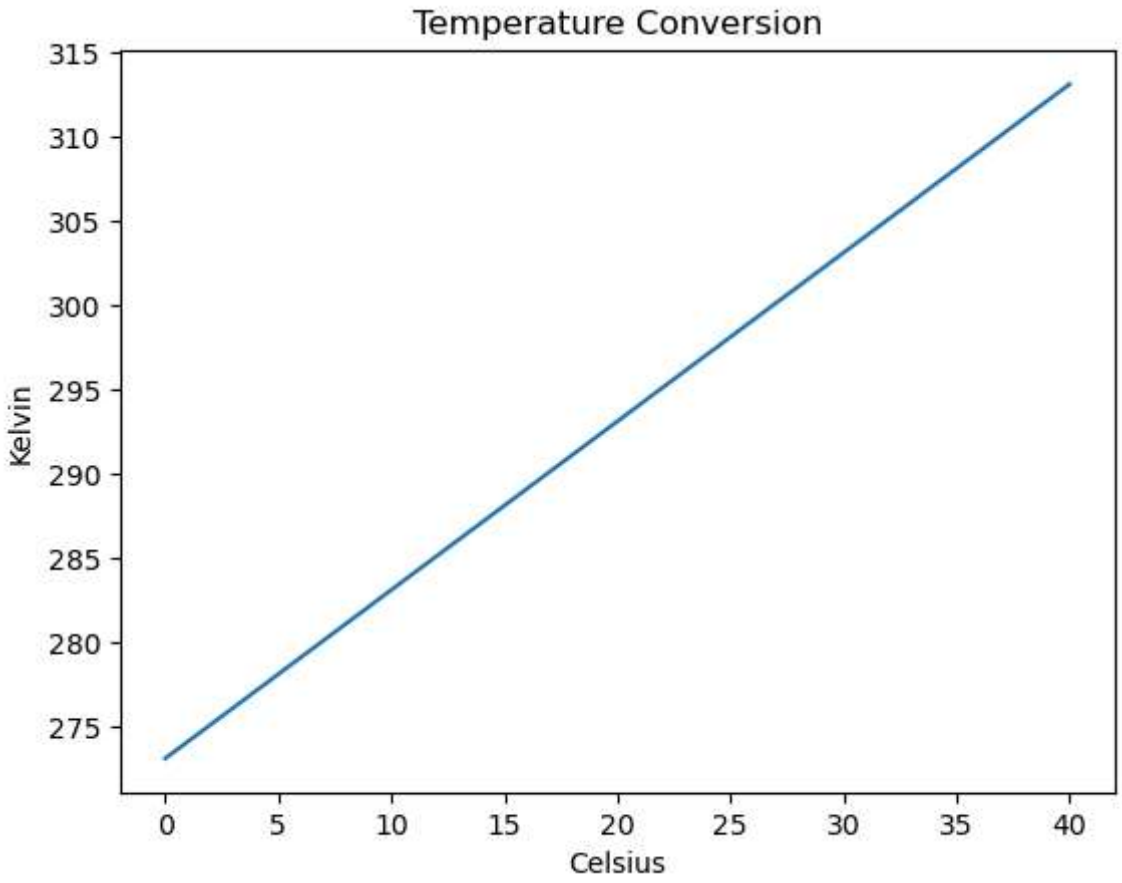
df = pd.DataFrame({"Celsius": celsius, "Kelvin": kelvin})

print(df)

plt.plot(df["Celsius"], df["Kelvin"])
plt.xlabel("Celsius")
plt.ylabel("Kelvin")
plt.title("Temperature Conversion")
plt.show()

"""Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52""
```

```
   Celsius  Kelvin
0        0  273.15
1       10  283.15
2       20  293.15
3       30  303.15
4       40  313.15
```



Out[23]: 'Name:- Pramathesh Kumar Sahoo, Regd.No:- 2341019043, Branch:- CSE, Sl.No-52'